

Hybrid Mobile Recommender System - Complete Project Plan

Project Overview

A comprehensive hybrid recommender system for mobile phones targeting Indian market with explainability, serendipity, contrarian recommendations, and bias detection features.

Technical Stack & Alternatives

1. Data Collection & APIs

Primary Sources:

- **GSMarena** (Web scraping with BeautifulSoup4)
- **91mobiles.com** (Backup source)
- **PhoneDB.net** (Additional specifications)

Alternatives if APIs fail:

- Kaggle datasets (GSMarena Phone Dataset)
- Manual CSV compilation from multiple sources
- Teoalida database (13,000+ models)

2. Backend Technologies

Primary Stack:

- **Python 3.9+** (Core language)
- **FastAPI** (API framework - faster than Flask)
- **SQLite/PostgreSQL** (Structured data)
- **FAISS** (Vector similarity search)
- **Pandas/NumPy** (Data processing)
- **Scikit-learn** (ML algorithms)

Alternatives:

- Flask instead of FastAPI
- ChromaDB instead of FAISS
- MongoDB for NoSQL approach

3. AI/ML Components

Recommendation Engine:

- **Sentence-Transformers** (Open source, no API limits)
- **BERT/DistilBERT** for text embeddings
- **Collaborative Filtering** (Matrix Factorization)
- **Content-based Filtering** (Feature similarity)

Explainability:

- **LIME** (Local Interpretable Model-agnostic Explanations)
- **SHAP** (SHapley Additive exPlanations)
- Custom rule-based explanations

4. Frontend Options

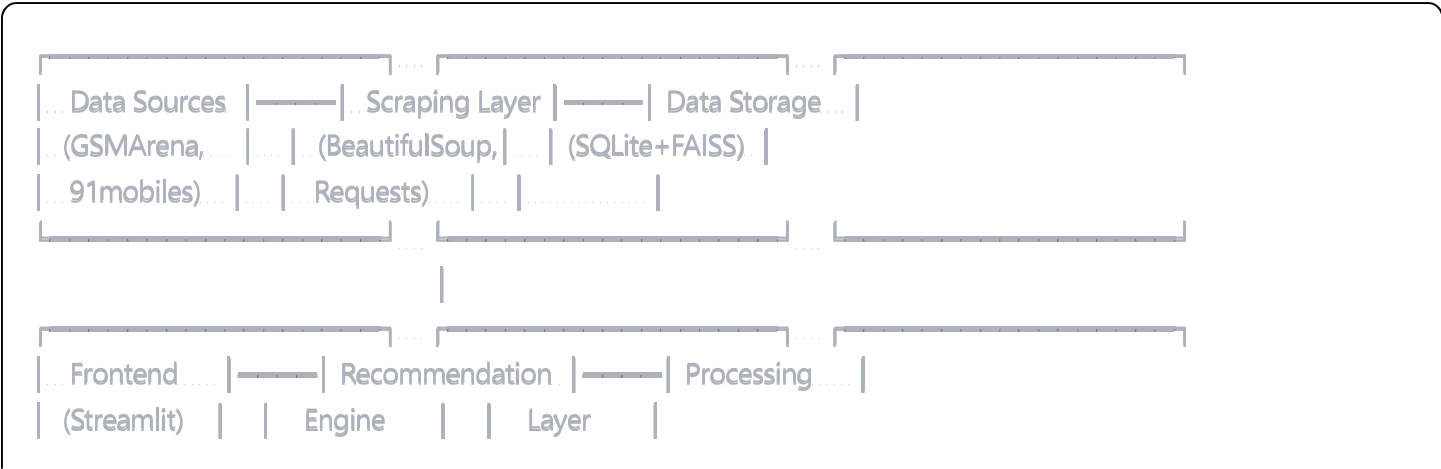
Primary Choice: **Streamlit** (Fastest development) **Alternatives:**

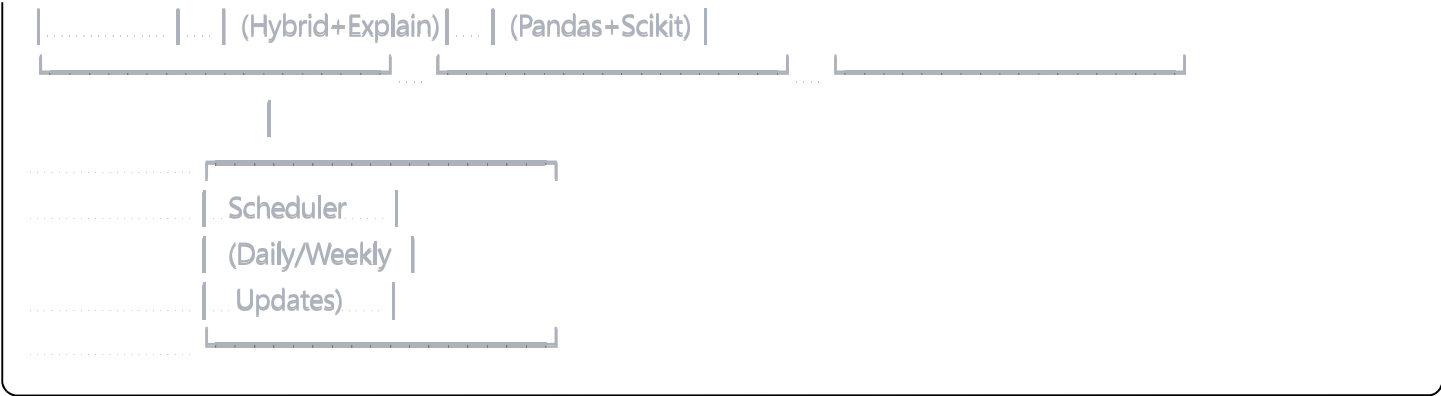
- Flask + Jinja2 templates
- React.js (if you prefer modern UI)
- Gradio (AI-focused UI)

5. Deployment

- **Local deployment** (as required)
- **Python virtual environment**
- **Requirements.txt** for dependencies
- **Simple batch scripts** for automation

System Architecture





Detailed Module Implementation

Module 1: Data Scraping Layer

Components:

- Multi-source scraper (GSMarena, 91mobiles, PhoneDB)
- Data validation and cleaning
- Price normalization across sources
- Specification standardization

Key Features:

```
python

class MobileScraper:
    - scrape_gsmarena()
    - scrape_91mobiles()
    - validate_specifications()
    - normalize_prices()
    - detect_duplicates()
    - calculate_derived_features()
```

Derived Features:

- Value for Money Score = (Spec Score × 100) / Price
- Battery to Weight Ratio = Battery Capacity / Weight
- Display Quality Index = (Resolution × PPI × Refresh Rate) / 1000
- Performance Index = (RAM × Storage × CPU Score) / 1000

Module 2: Scheduler Layer

Purpose: Automated data updates and maintenance

Functions:

- Daily price updates
- Weekly specification updates
- Monthly full data refresh
- Data quality monitoring
- Bias detection alerts

python

```
class DataScheduler:
... - schedule_daily_updates()
... - monitor_data_quality()
... - detect_pricing_anomalies()
... - update_popularity_scores()
```

Module 3: Data Storage Layer

Structure:

sql

```
-- Main phones table
CREATE TABLE phones (
... id INTEGER PRIMARY KEY,
... brand TEXT,
... model TEXT,
... launch_date DATE,
... price REAL,
... specs_json TEXT,
... rating REAL,
... popularity_score REAL
);

-- User interactions
CREATE TABLE user_interactions (
... user_id TEXT,
... phone_id INTEGER,
... interaction_type TEXT,
... timestamp DATETIME
);
```

Vector Storage (FAISS):

- Phone feature embeddings
- User preference vectors
- Similarity indices

Module 4: Processing Layer

Data Cleaning Pipeline:

1. Handle missing values
2. Standardize units (GB, MP, mAh, etc.)
3. Normalize categorical features
4. Detect and handle outliers
5. Feature engineering

Quality Checks:

- Price validation against market trends
- Specification consistency
- Brand standardization
- Duplicate detection

Module 5: Recommendation Engine

Hybrid Approach:

1. **Collaborative Filtering (40%)**
 - User-item matrix factorization
 - Similar user preferences
2. **Content-based Filtering (40%)**
 - Feature similarity using cosine similarity
 - Specification matching
3. **Popularity Boost (10%)**
 - Recent launch bonus
 - User rating consideration
4. **Diversity Factor (10%)**
 - Serendipity injection

- Brand diversity

Explainability Components:

python

```
class ExplainableRecommender:
    def explain_recommendation(self, phone_id, user_preferences):
        return {
            "why_recommended": "High camera quality matches your photography needs",
            "feature_match": {"camera": 0.95, "battery": 0.87, "performance": 0.82},
            "similar_phones": [list_of_alternatives],
            "unique_selling_points": ["Best camera in price range", "5G enabled"],
            "contrarian_aspect": "Lesser-known brand but superior specs"
        }
```

Serendipity Implementation:

- 20% weight to phones from non-top-5 brands
- Boost phones with unique features
- Penalize over-popular recommendations

Contrarian Logic:

- Identify phones going against trends (e.g., headphone jack when others remove it)
- Small brand phones with flagship features
- Older models with better value propositions

Module 6: Bias Detection

Detection Mechanisms:

1. Brand Over-representation

- Monitor brand distribution in recommendations
- Alert if >30% recommendations from single brand

2. Price Bias

- Track recommendation price distribution
- Ensure budget options aren't ignored

3. Specification Bias

- Monitor if certain specs are over-weighted

- Balance between different user priorities

4. **Temporal Bias**

- Prevent only recent phones being recommended
- Include value-oriented older models

Module 7: Frontend Layer (Streamlit)

User Interface Components:

- User preference selection
- Search and filter options
- Recommendation display with explanations
- Comparison tool
- User feedback collection

Key Pages:

1. **Home/Preference Setting**
2. **Recommendations with Explanations**
3. **Phone Comparison**
4. **Serendipity Discoveries**
5. **Contrarian Suggestions**
6. **User Profile/History**

Implementation Challenges & Solutions

Challenge 1: Data Quality & Consistency

Problem: Different sources have varying formats, missing data **Solution:**

- Multi-source validation
- Confidence scoring for each data point
- Manual verification for critical specs
- Fallback mechanisms

Challenge 2: Real-time Price Updates

Problem: Prices change frequently across platforms **Solution:**

- Daily automated scraping

- Price history tracking
- Alert system for major changes
- Multiple source validation

Challenge 3: API Rate Limits

Problem: Websites may block excessive requests **Solution:**

- Respectful scraping with delays
- Rotating user agents
- Multiple data sources
- Cached data fallback

Challenge 4: Recommendation Quality

Problem: Cold start, sparse data, bias **Solution:**

- Hybrid approach combining multiple methods
- Content-based fallback for new users
- Popularity-based initial recommendations
- Active learning from user feedback

Challenge 5: Explainability Accuracy

Problem: Ensuring explanations match actual recommendation logic **Solution:**

- Direct feature importance extraction
- Rule-based explanation generation
- Confidence scores for explanations
- User feedback on explanation quality

Development Timeline (8-10 weeks)

Week 1-2: Data Collection & Storage

- Implement scraping modules
- Set up database schema
- Create data validation pipeline
- Test with sample data

Week 3-4: Processing & Feature Engineering

- Build data cleaning pipeline
- Implement derived feature calculations
- Create data quality monitoring
- Set up scheduler system

Week 5-6: Recommendation Engine

- Implement collaborative filtering
- Build content-based filtering
- Create hybrid combination logic
- Add explainability components

Week 7: Bias Detection & Advanced Features

- Implement serendipity logic
- Add contrarian recommendations
- Build bias detection systems
- Create user interaction tracking

Week 8: Frontend & Integration

- Build Streamlit interface
- Integrate all backend components
- Add comparison features
- Implement user authentication

Week 9-10: Testing & Optimization

- End-to-end testing
- Performance optimization
- User acceptance testing
- Documentation completion

File Structure

```
mobile_recommender/  
├── src/
```

```
|...|— scrapers/
|...|...|— gsmarena_scraper.py
|...|...|— mobiles91_scraper.py
|...|...|— base_scraper.py
|...|— data/
|...|...|— storage.py
|...|...|— preprocessing.py
|...|...|— feature_engineering.py
|...|— recommender/
|...|...|— collaborative.py
|...|...|— content_based.py
|...|...|— hybrid.py
|...|...|— explainer.py
|...|— bias_detection/
|...|...|— brand_bias.py
|...|...|— price_bias.py
|...|...|— spec_bias.py
|...|— scheduler/
|...|...|— data_updater.py
|...|— frontend/
|...|...|— streamlit_app.py
|— data/
|...|— raw/
|...|— processed/
|...|— vectors/
|— models/
|— config/
|— tests/
|— docs/
|— requirements.txt
|— README.md
```

Key Libraries & Dependencies

python

```
# requirements.txt
streamlit==1.28.0
fastapi==0.104.0
pandas==2.1.0
numpy==1.24.0
scikit-learn==1.3.0
beautifulsoup4==4.12.0
requests==2.31.0
sqlite3 # Built-in
faiss-cpu==1.7.4
sentence-transformers==2.2.2
lime==0.2.0.1
shap==0.42.0
plotly==5.17.0
python-multipart==0.0.6
schedule==1.2.0
python-dotenv==1.0.0
```

Success Metrics

1. **Recommendation Accuracy:** >85% user satisfaction
2. **Diversity:** Max 30% from any single brand
3. **Data Coverage:** >95% of Indian market phones
4. **Response Time:** <2 seconds for recommendations
5. **Explanation Quality:** >80% users find explanations helpful
6. **Serendipity Success:** 15% users discover new brands
7. **Bias Detection:** <5% false positive rate

Risk Mitigation

1. **Data Source Blocking:** Multiple backup sources
2. **Quality Issues:** Robust validation pipeline
3. **Performance Problems:** Caching and optimization
4. **User Adoption:** Intuitive UI and clear value proposition
5. **Maintenance Burden:** Automated monitoring and alerts

Future Enhancements

1. **Real-time Notifications:** Price drops, new launches

2. **Social Features:** Community reviews, sharing
3. **Advanced Analytics:** Market trend analysis
4. **Mobile App:** React Native implementation
5. **API Endpoints:** Third-party integration
6. **Machine Learning:** Advanced neural networks

This comprehensive plan addresses all your requirements while providing practical solutions for the challenges you'll face. Would you like me to start implementing any specific module first?